SVM
PCA

Vladislav Goncharenko
Senior researcher

girafe ai

ITMO, fall 2026

Recap

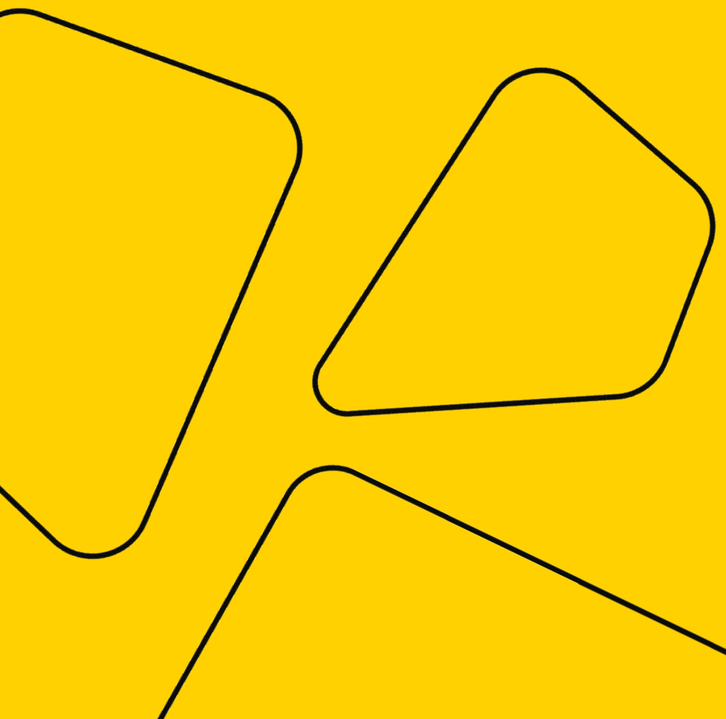
Lecture 3:
Logistic Regression
Classification metrics



1. Linear classification
    a. margin
    b. loss functions
2. Logistic Regression
    a. sigmoid derivation
    b. MLE for LogReg
    c. logistic loss
    d. probability calibration
3. Multiclass aggregation strategies
    a. One vs Rest
    b. One vs One
4. Metrics in classification
    a. Accuracy, Balanced accuracy
    b. Precision, Recall, F-score
    c. ROC curve, PR curve, AUC
    d. Confusion matrix
5. Hardware for ML

Outline

Lecture 4:
Logistic Regression
Classification metrics



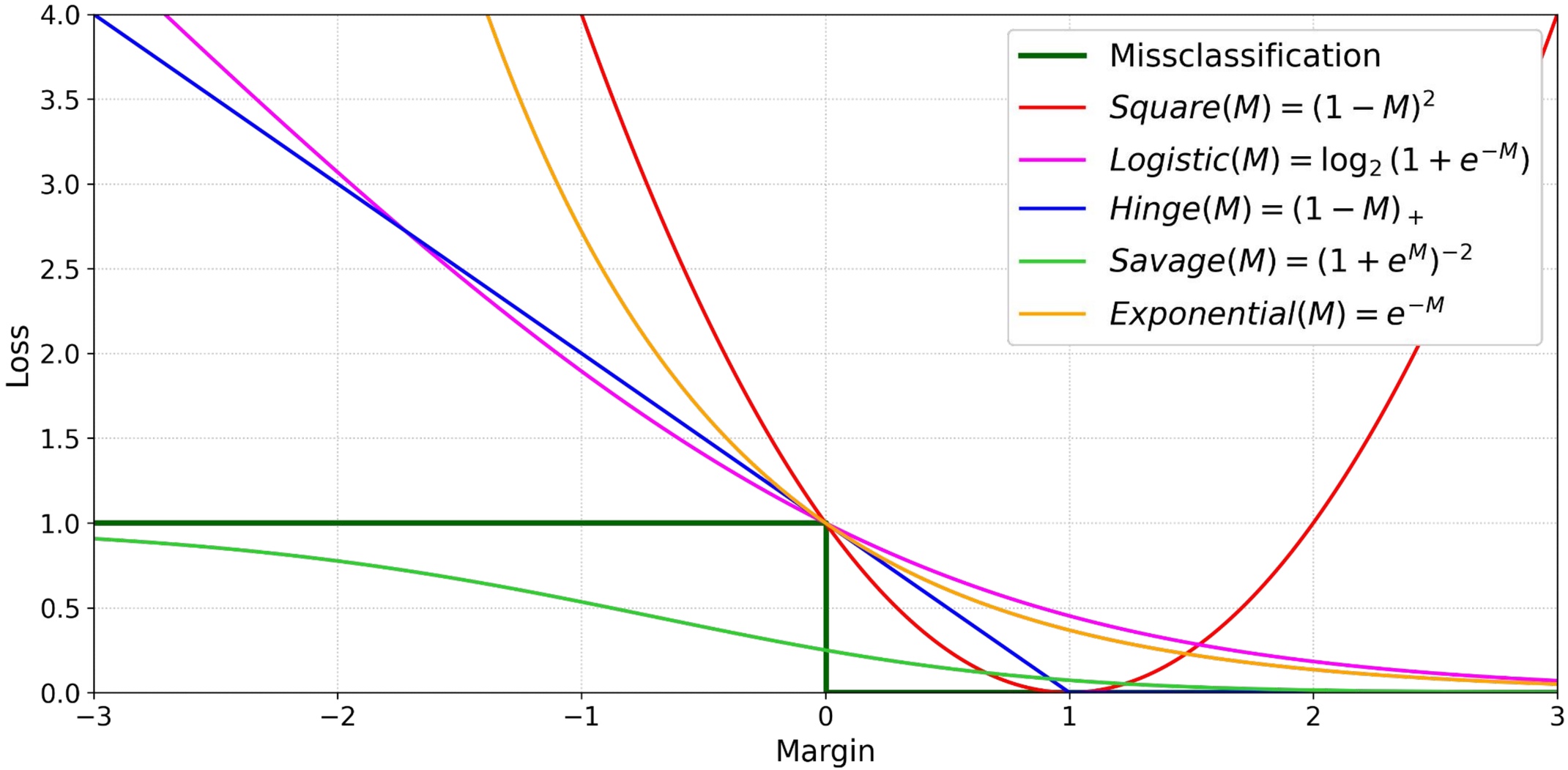
1. Support Vector Machine (SVM)
    a. Hinge loss
    b. Kernel trick
2. Dimensionality reduction and PCA
    a. Problem statement
    b. Singular Value Decomposition
    c. Eckart–Young theorem
    d. Equivalent definitions
    e. Data normalization

Support Vector Machine

girafe ai

01

Classification problem losses



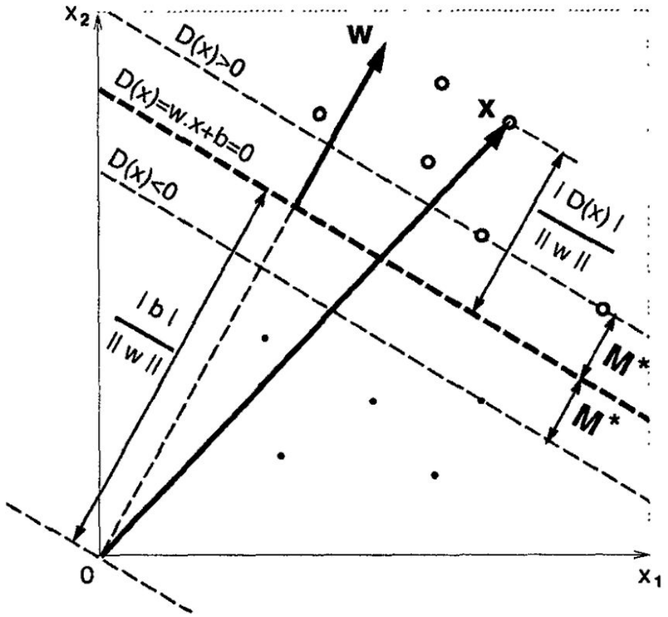
Loss functions for classification

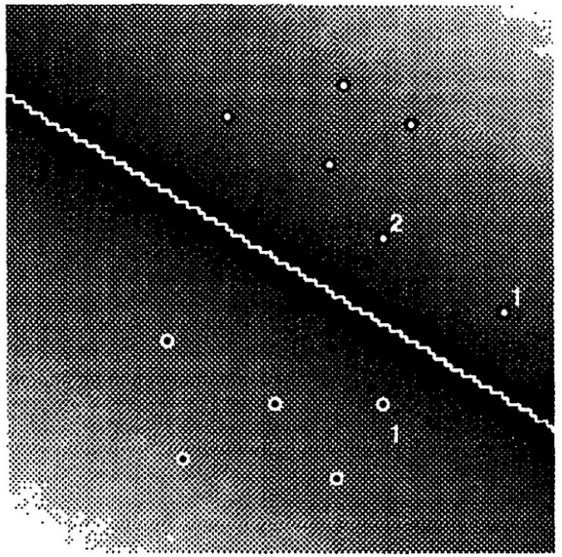
Support Vector Machine



1. History
2. Motivation
3. Solution for separable design
4. Inseparable design, soft margin
5. Kernels
    a. Kernel definition (Hilbert spaces, inner product, positive semidefiniteness)
    b. Kernels properties (addition, infinite sums)
    c. Types of kernels (poly, exponential, gaussian)
6. Current state

History





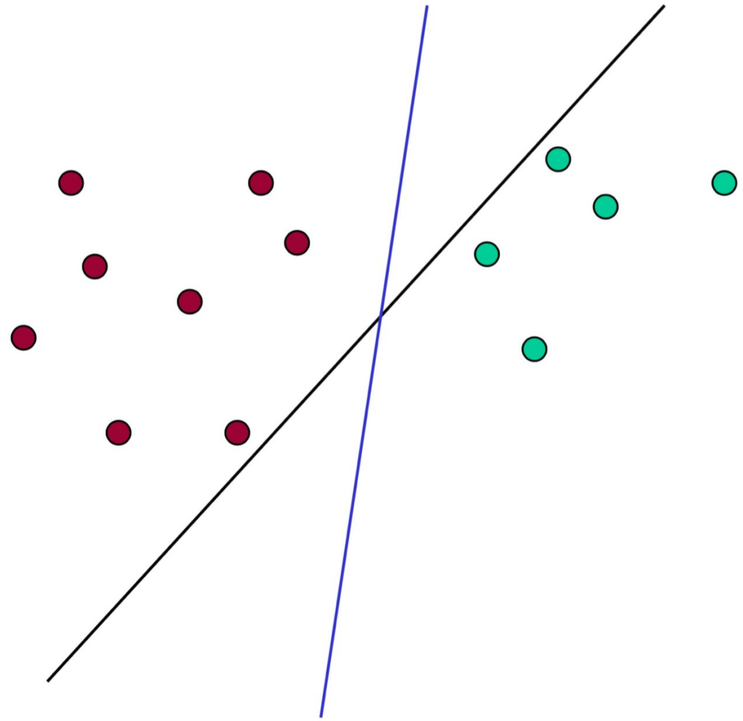
1963: SVM introduced by Soviet mathematicians
Vladimir Vapnik and Alexey Chervonenkis (article 1, article 2)

1992: kernel trick (Vapnik, Boser, Guyon)

1995: soft margin (Vapnik, Cortes)

More on Vapnik, more on Chervonenkis

Motivation



Linear separable case

Many separating hyperplanes exist

Maximize width

Margin

$$y \in \{1, -1\}$$

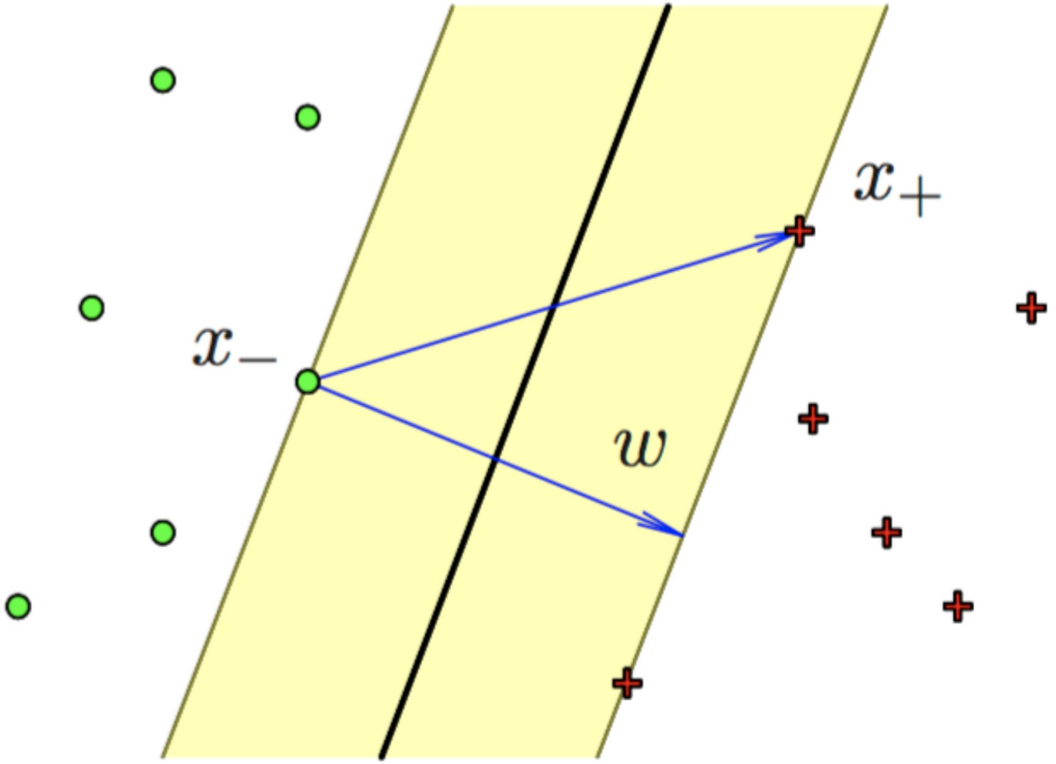
$$y_i = 1 : w^T x_i - c > 0$$

$$y_i = -1 : w^T x_i - c < 0$$

$$c_+(w) = \min_{y_i=1}(w^T x_i)$$

$$c_-(w) = \max_{y_i=-1}(w^T x_i)$$

$$\rho(w) = \frac{c_+(w) - c_-(w)}{2}$$



$$\rho\left(\frac{w_0}{||w_0||}\right) = \frac{1}{||w_0||}$$

Optimization problem

$$y_i = 1 : w^T x_i - c > 0$$
$$y_i = -1 : w^T x_i - c < 0$$
$$M_i = y_i \cdot (w^T x_i - c)$$

$$\rho(w) = \frac{1}{||w||} \to \max_{w,c}$$
$$s.t.\ y_i(w^T x_i - c) \geq 1$$

Convex problem!

$$L(w, c, \alpha) = \frac{1}{2} w^T w - \sum_i \alpha_i (y_i (w^T x_i - c) - 1)$$

Many of them are zeros

Hinge loss

$$L(w, c, \alpha) = \frac{1}{2} w^T w - \sum_i \alpha_i (y_i (w^T x_i - c) - 1)$$

$$L^{\text{hinge}} = (1 - M)_+$$

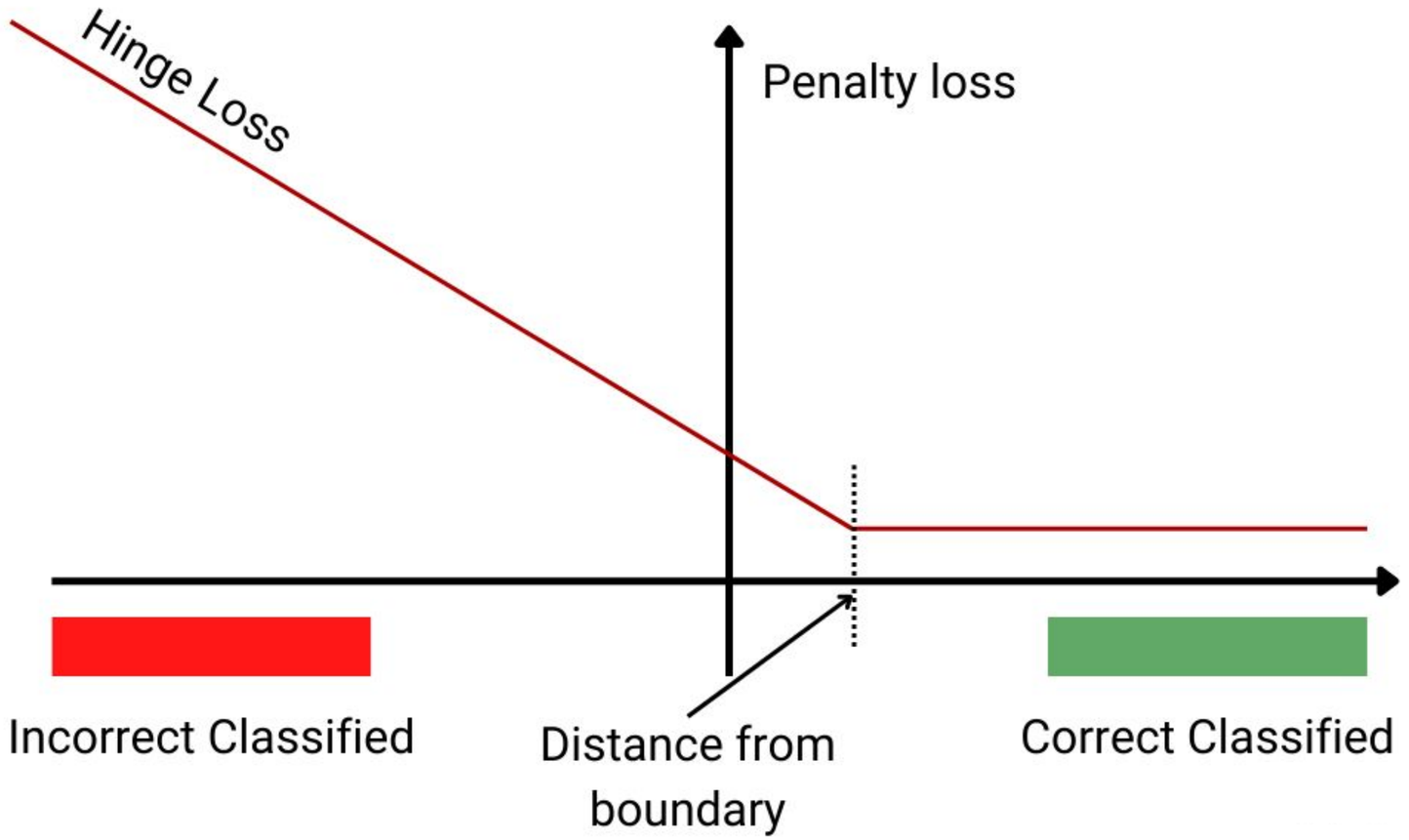
$$L(w, c, \alpha) = \frac{1}{2} ||w||_2^2 + \sum_i \alpha_i L_i^{\text{hinge}}$$

Loss is retrieved from “subject to” part. Positive part notation.

Here “regularization” term is responsible for stripe width.

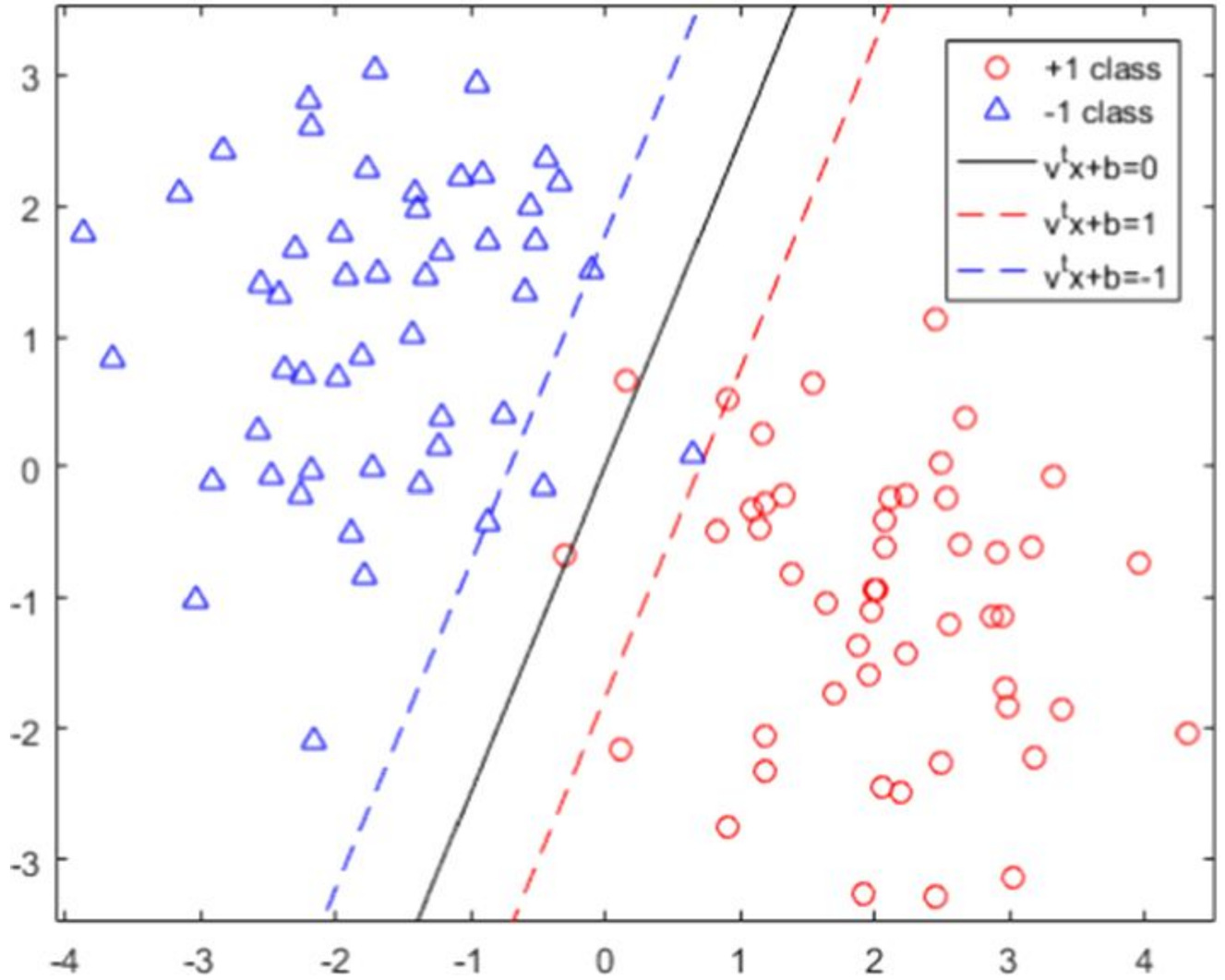
Hinge loss

$$L^{\text{hinge}} = (1 - M)_+$$



Inseparable case

Let our model mistake, but penalize that mistakes



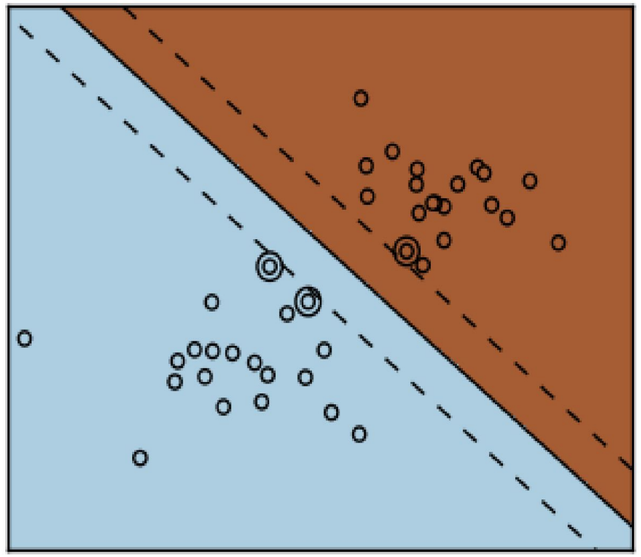
Inseparable case

Let our model mistake, but penalize that mistakes

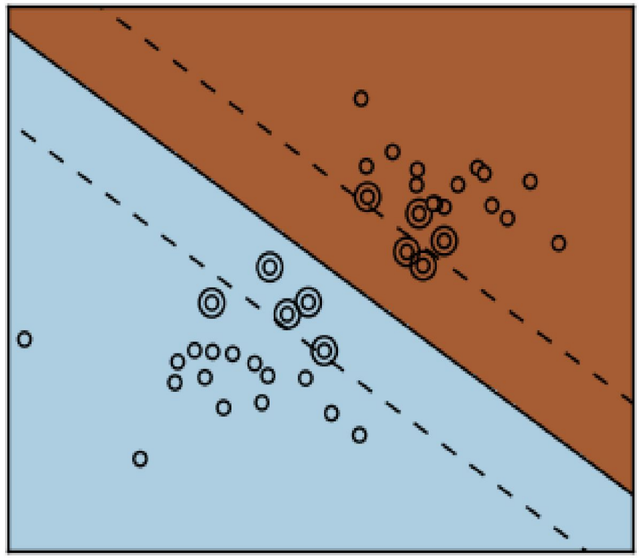
Implemented via margin slack variables

$$\begin{cases} \dfrac{1}{2}\langle w, w\rangle + C\sum\limits_{i=1}^{\ell}\xi_i \to \min\limits_{w, w_0, \xi}; \\ y_i\big(\langle w, x_i\rangle - w_0\big) \geqslant 1 - \xi_i, \quad i = 1, \ldots, \ell; \\ \xi_i \geqslant 0, \quad i = 1, \ldots, \ell. \end{cases}$$

Big C



Small C



Kernel trick

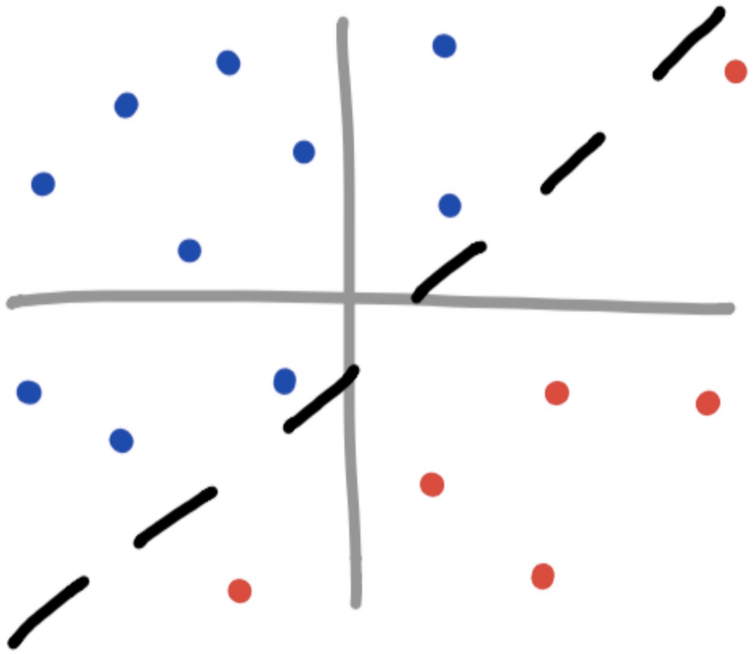
$$y_i = 1 : w^T x_i - c > 0$$
$$y_i = -1 : w^T x_i - c < 0$$

$$\begin{matrix} x \mapsto \phi(x) \\ w \mapsto \phi(w) \end{matrix} \Longrightarrow < w, x > \mapsto < \phi(w), \phi(x) >$$

$$K(w, x) =< \phi(w), \phi(x) >$$

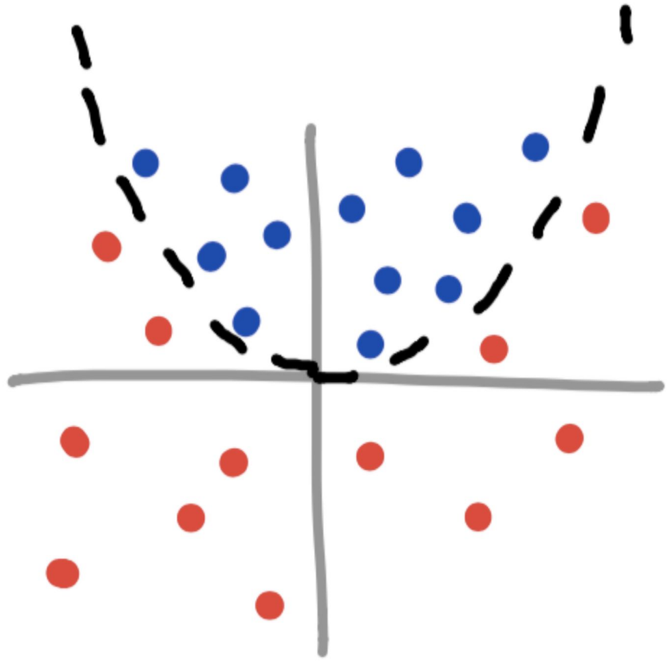
Visual explanation
https://www.youtube.com/watch?v=Q7vT0--5VII

Kernel types



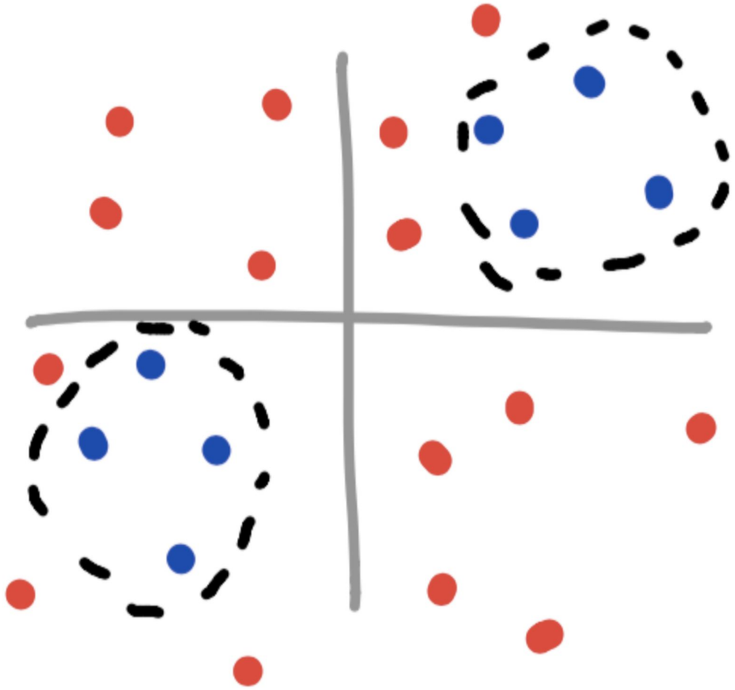
$K(w,x) =< \mathrm{w}, x >$

Linear



$K(w,x) = (\gamma < w,x > +r)^d$

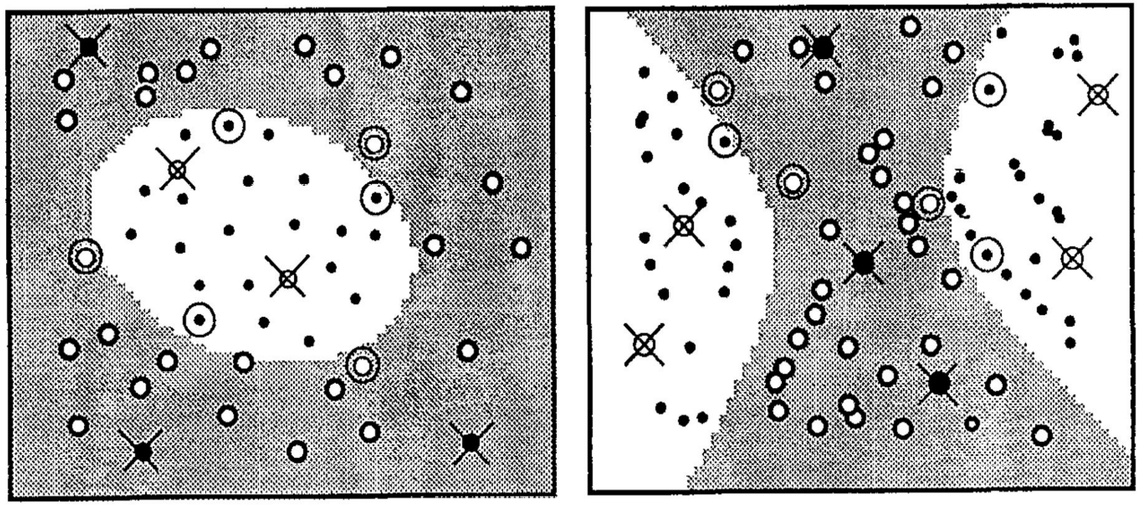
Polynomial



$K(w,x) = e^{-\gamma\|w-x\|^2}$

Gaussian
radial basis function

Current state



90s-00s

2010s



The bitter lesson

Rich Sutton, author of many Reinforcement Learning algorithms, wrote an article in 2019.

“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin”

Methods have to be scalable such that:

- benefit from more compute
- benefit from more data

otherwise they will be discarded by Moore’s law in relatively short term.

Original text, Wikipedia



Matrix norms

girafe ai

00

Matrix norms



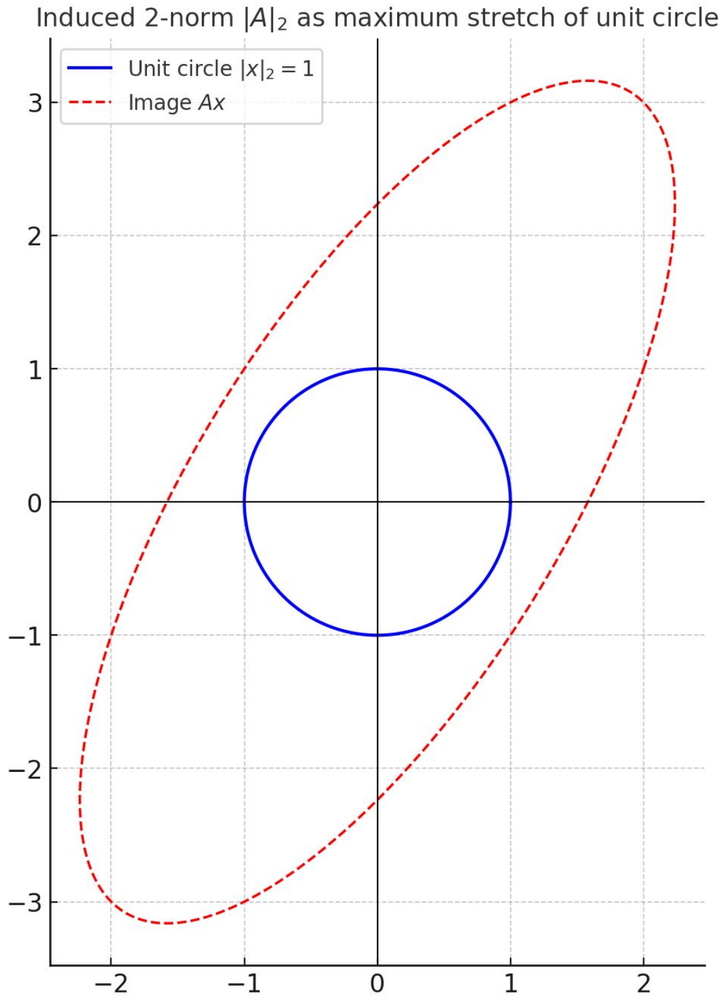
For many cases in math we need to measure matrices, so we introduce norm term for matrix (wiki):

1. Nonnegativity: $\|A\| \geq 0$
2. Definiteness: $\|A\| = 0 \iff A = 0_{m,n}$
3. Absolute homogeneity: $\|\alpha A\| = |\alpha| \, \|A\|$
4. Triangle inequality: $\|A + B\| \leq \|A\| + \|B\|$
5. Submultiplicativity: $\|AB\| \leq \|A\| \, \|B\|$

Matrix p-norms

Especially useful are vector induced norms, they are defined as

$$\|A\|_p = \sup_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p} = \sup_{\|x\|_p = 1} \|Ax\|_p$$



Matrix p-norms examples

Some corner cases:

- p = 2: spectral norm
- p = 1: maximum absolute column sum norm
- p = ∞: maximum absolute row sum norm
- p = 0: maximum number of nonzero entries per column

$$\|A\|_2 = \sup_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sigma_{\max}(A)$$

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^{m} |a_{ij}|$$

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^{n} |a_{ij}|$$

$$\|A\|_0 = \max_{1 \leq j \leq n} \sum_{i=1}^{m} \mathbf{1}_{\{a_{ij} \neq 0\}}$$

Principal Component Analysis

girafe ai

02

Dimensionality reduction problem

$$x_1, \ldots, x_n \rightarrow g_1, \ldots, g_k, k \leq n$$

Principal Component Analysis

$$x_1, \ldots, x_n \rightarrow g_1, \ldots, g_k, k \leq n$$

$$U : UU^T = I, G = XU$$

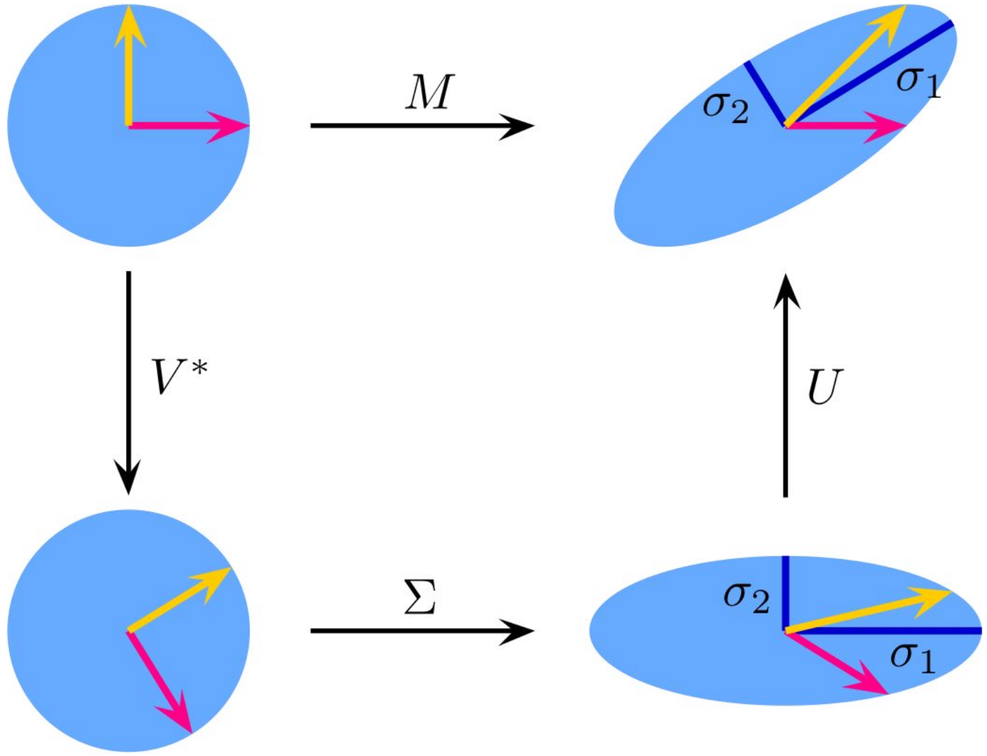
$$\hat{X} = GU^T \approx X$$

$$||GU^T - X|| \rightarrow \min_{G,U} \; s.t. \; rank(G) \leq k$$

Singular value decomposition

factorization of a real or complex matrix into a rotation, followed by a rescaling followed by another rotation.

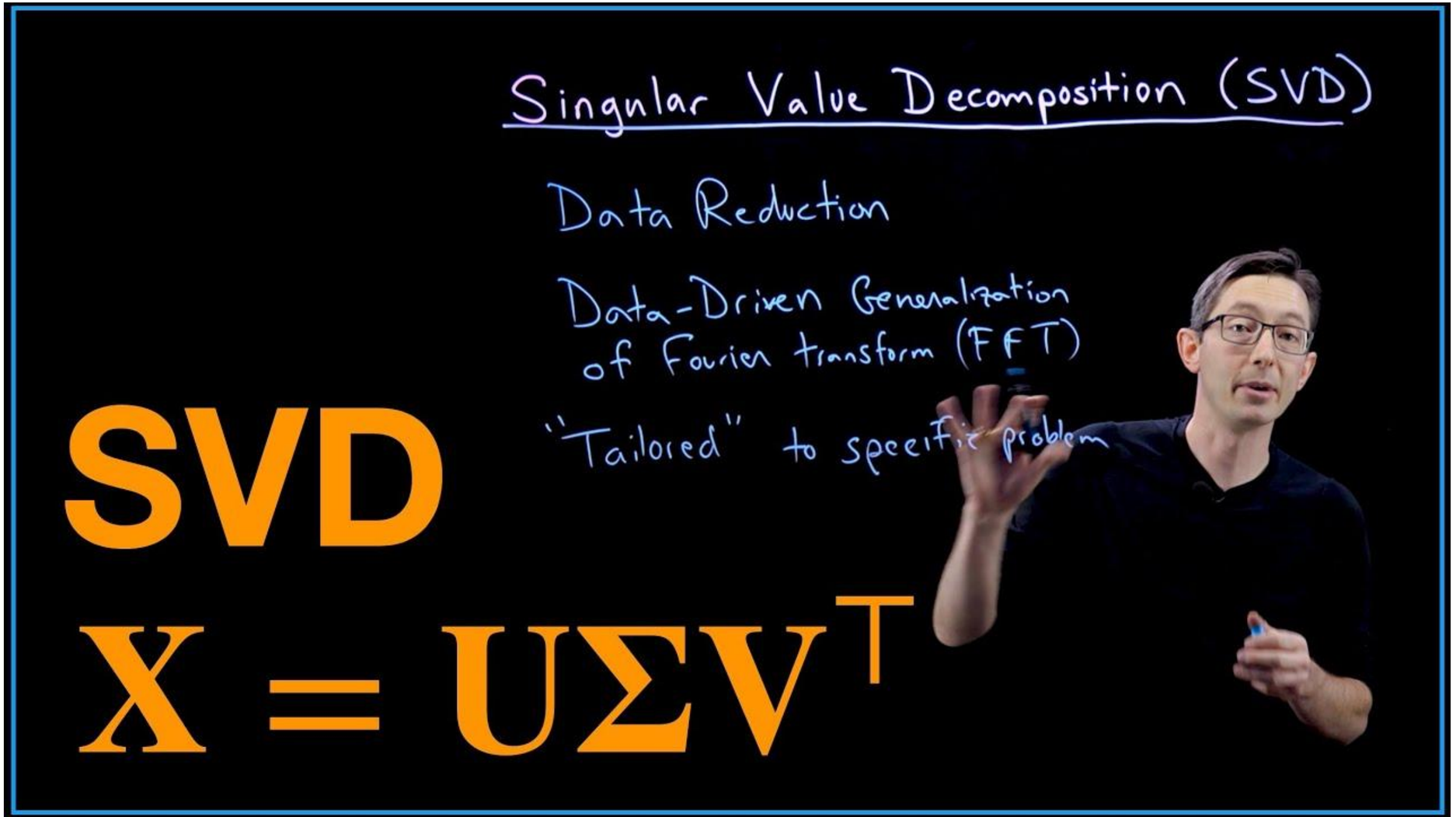
It generalizes the eigendecomposition of a square normal matrix with an orthonormal eigenbasis to any m×n matrix. It is related to the polar decomposition.



$$M = U \cdot \Sigma \cdot V^*$$

SVD explained

https://www.youtube.com/playlist?list=PLMrJAkhIeNNSVjnsviglFoY2nXildDCcv



PCA derivation

$$||GU^T - X|| \rightarrow \min_{G,U} \; s.t.\; rank(G) \leq k$$

$$X = V\Sigma U^T : ||GU^T - V\Sigma U^T||_2 = ||G - V\Sigma||_2$$

$$G = V\Sigma' : ||V\Sigma' - V\Sigma||_2 = ||\Sigma' - \Sigma||_2$$

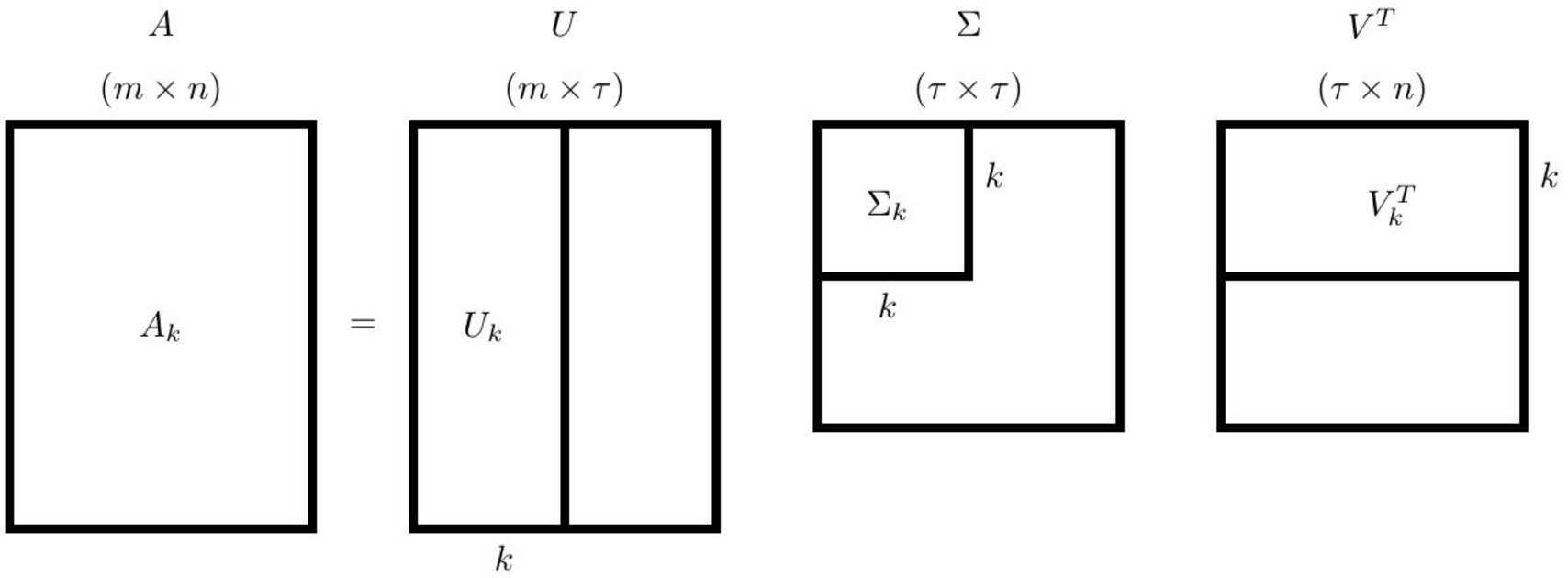
$$||A||_2 = \sigma_{max}(A) : ||\Sigma' - \Sigma||_2 = \sigma_k(\Sigma) = \sigma_k(X)$$

Eckart–Young–Mirsky theorem

PCA as SVD

$$||GU^T - X|| \to \min_{G,U} \; s.t.\; rank(G) \leq k$$

$$X = V\Sigma U^T \qquad \sigma_k(\Sigma) = \sigma_k(X)$$



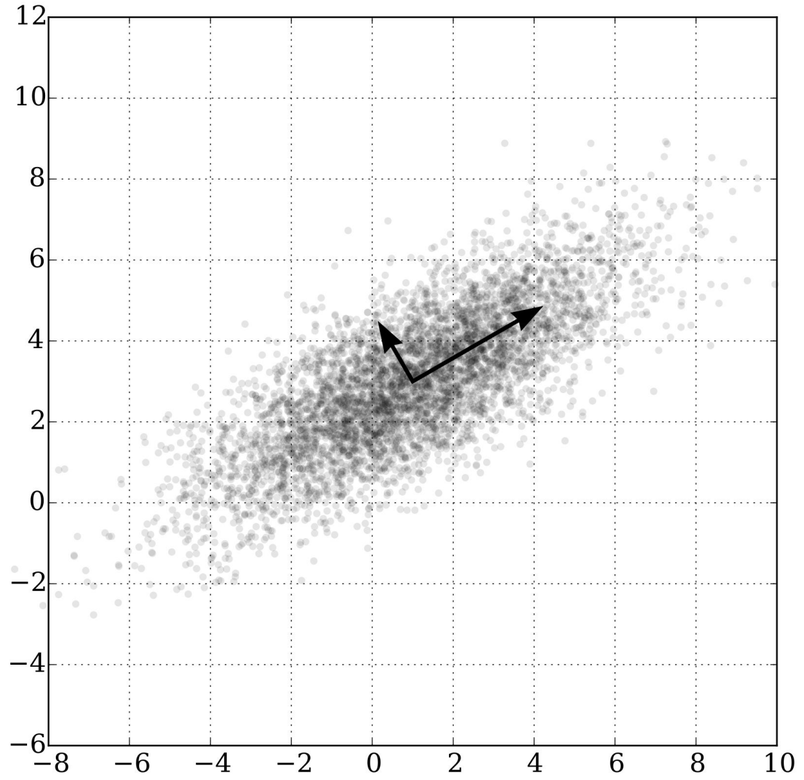
Eckart–Young–Mirsky theorem

Variance based PCA algorithm

1. Choose direction with max projected variance, repeat till enough components
2. Remove direction with least projected variance, repeat till enough components remain

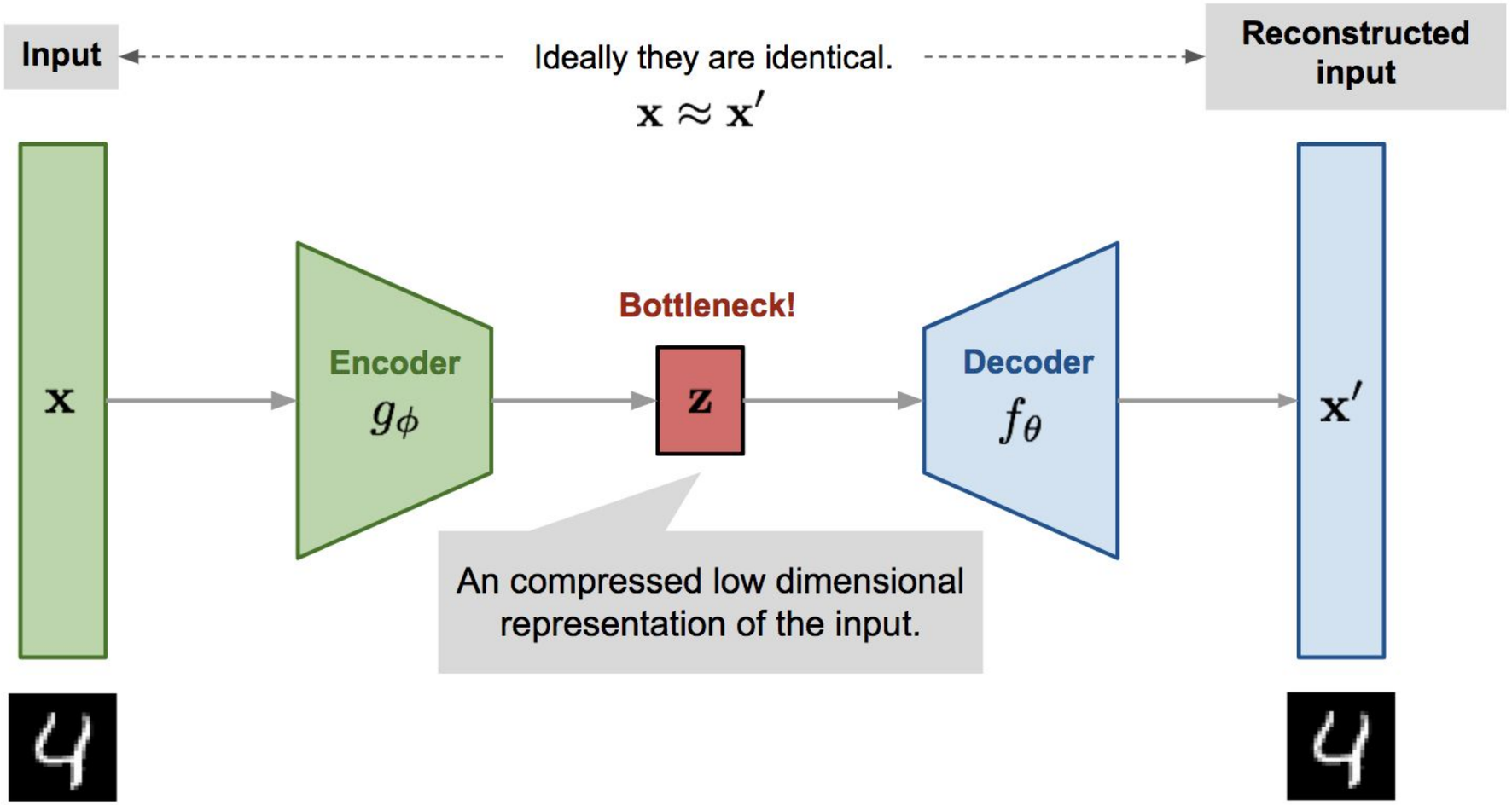
Take new basis vectors greedy

Same result for G and U

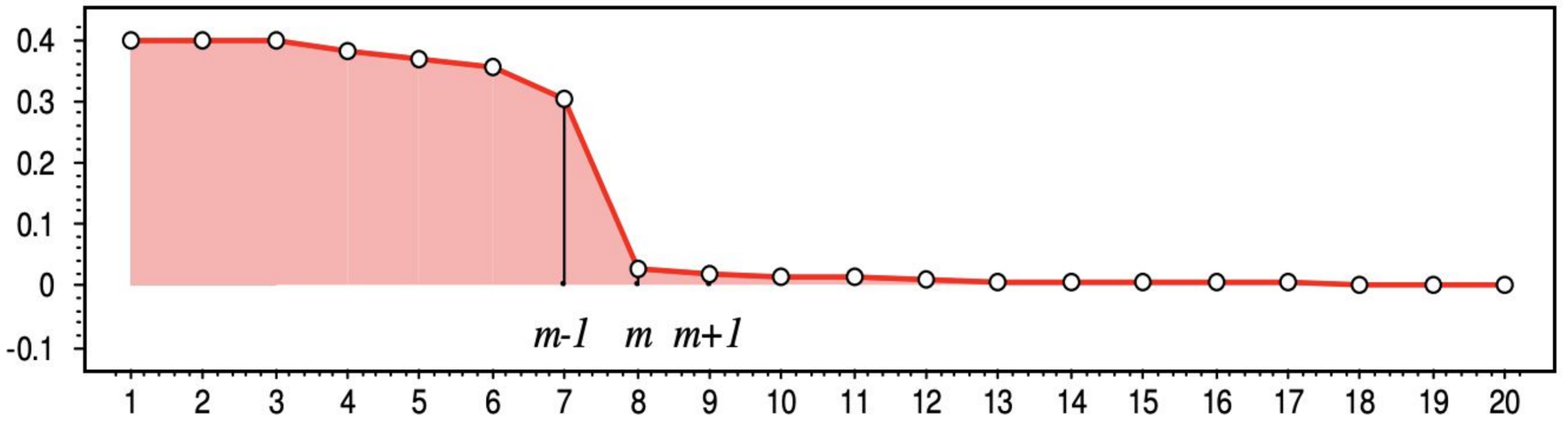


PCA as autoencoder

https://en.wikipedia.org/wiki/Autoencoder



Relative variance threshold



Get rid of low-variance components

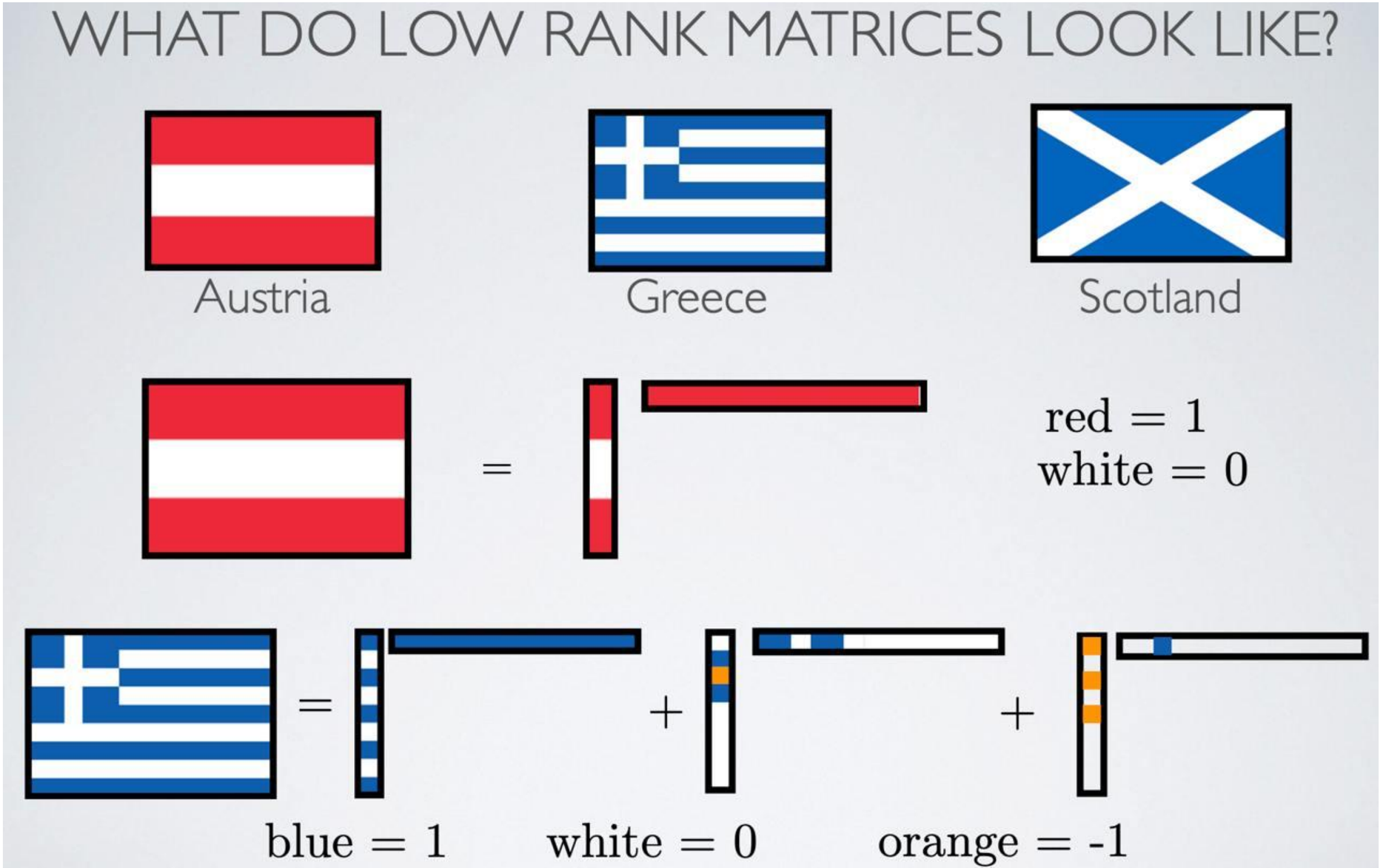
$$E_m = \frac{\|GU^\intercal - F\|^2}{\|F\|^2} = \frac{\lambda_{m+1} + \cdots + \lambda_n}{\lambda_1 + \cdots + \lambda_n} \leqslant \varepsilon.$$

PCA for visualization



Let’s walk through space…

PCA on images



SVD on images



16 base components

Dimensionality reduction



50 components restoration

Dimensionality reduction





250 components restoration

Problem with PCA



Flat price prediction

Features:

- square (sq. meters)
- distance to downtown (meters)
- ...

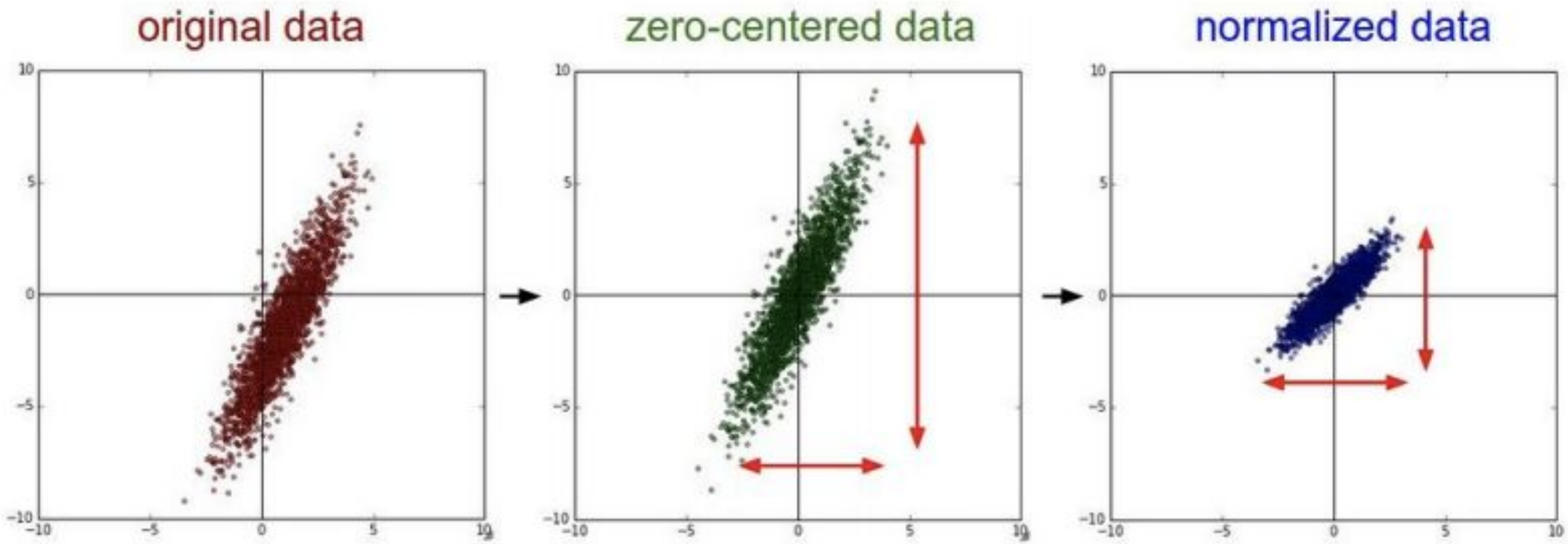
Solution: normalize data

True for any linear model!


линейные модели
$\hat{y} = x^T w + b$
StandardScaler
sklearn
$z = \frac{x - \mu}{\sigma}$
друеют с этой прикормки

Data normalization

Always normalize data before PCA!!!



Normalization types

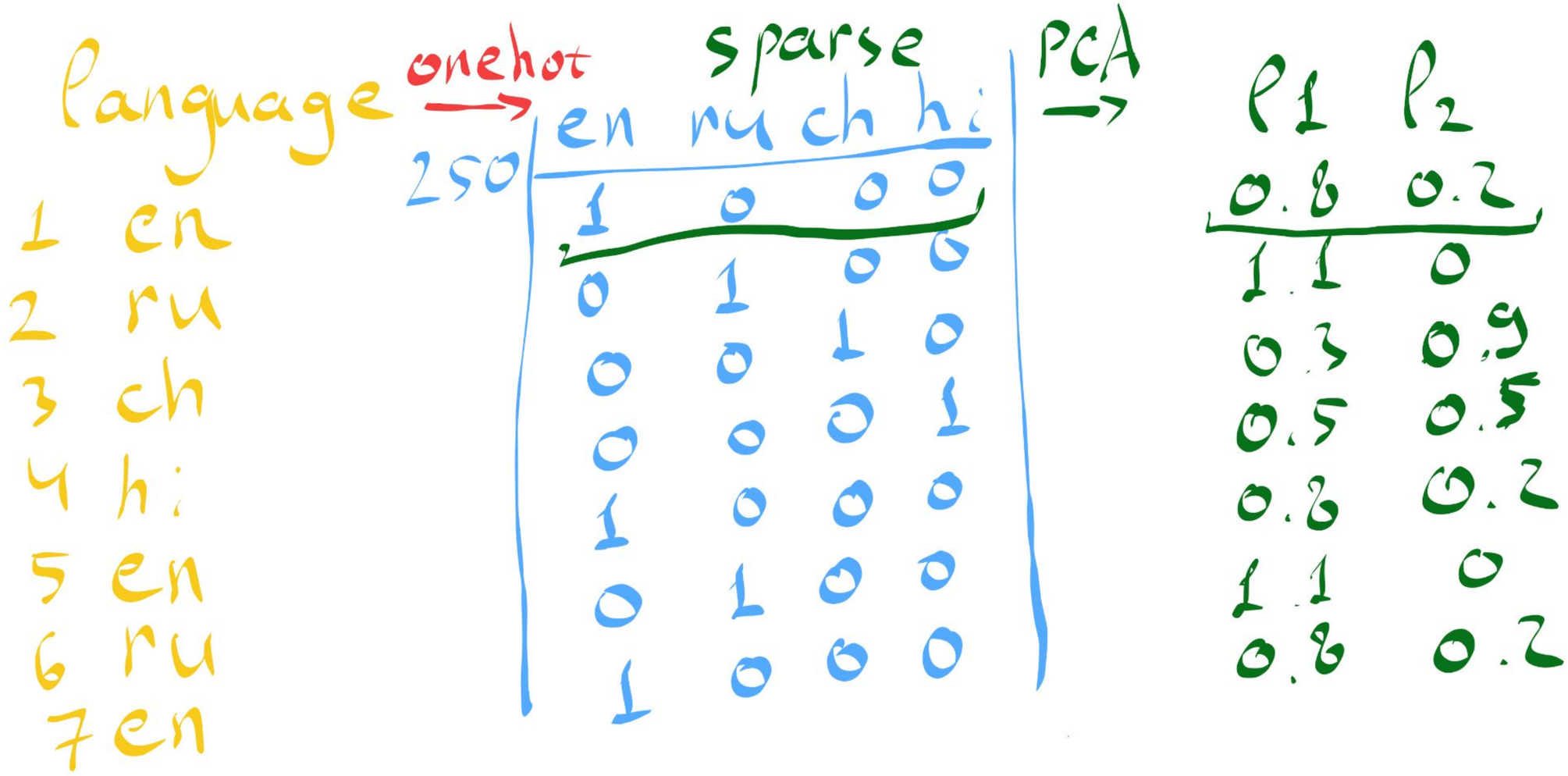


To follow sklearn notation:

- MinMaxScaler
  - scales to 0 to 1 range
- StandardScaler
  - removes mean and std from values distribution

PCA on one-hot encoding

For categorical feature we can obtain categorical vectors if we apply PCA on one-hot columns of that feature. See [Dyakonov's lecture]



Revise

1. Support Vector Machine (SVM)
    a. Hinge loss
    b. Kernel trick
2. Dimensionality reduction and PCA
    a. Problem statement
    b. Singular Value Decomposition
    c. Eckart–Young theorem
    d. Equivalent definitions
    e. Data normalization

Next time



1. k Nearest Neighbors model
    a. contemporary use cases
2. kNN indexes
    a. ideas
    b. Annoy
    c. HNSW
3. Implementations
    a. FAISS
4. Vector databases
    a. Qdrant

Thanks for attention!

Questions?



